



Security Assessment

Final Report



Umia Chainbound

April 2026

Prepared for Umia

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	9
Severity Matrix.....	9
Detailed Findings.....	10
High Severity Issues.....	13
H-01: TWAP initial snapshot dilution.....	13
H-02: User CPMM LP Permanently Lost at Settlement.....	16
Medium Severity Issues.....	19
M-01: migrate() always fails when currency is ETH because UmiaLBP cannot accept native ETH.....	19
M-02: Updating a QOrg's position operator via approveQOrgPositionOperator silently breaks all pending decision markets.....	21
M-03: Assets might stuck permanently after liquidation starts.....	22
M-04: A hacked Hub owner can steal assets from any QOrg.....	24
M-05: Leftover tokens stuck permanently in UmiaMarketManager after re-adding liquidity to Uniswap V4 during market settlement.....	26
Low Severity Issues.....	29
L-01: Paused QOrg Token blocks liquidation claims after liquidation starts.....	29
L-02: Migration uses fixed reserveTokenAmount and ignores swept unsold tokens in calculating liquidity to initialize UniswapV4 Pool.....	30
L-03: TWAP cardinality slot filling enables permissionless DoS on market creation and settlement.....	32
L-04: LP NFT withdrawal permanently breaks market creation.....	35
L-05: Executor's `ActionType.CALL` enables unauthorized address(0) upgrade and beacon reset.....	37
L-06: Conditional market swap applies two fee layers and effective fee exceeds conditionalMarketFeeBps.....	40
L-07: Threshold and config changes affect active markets.....	42
L-08: Unbounded string lengths in `uploadDocument` allow for permanent state bloat and gas limit DoS.....	44
L-09: A single reverting action can permanently DoS an entire governance execution plan.....	46
L-10: Duplicate assets in `LIQUIDATE_TREASURY` payloads allow users to drain the treasury via multiple payouts.....	47
L-11: CreateMarket does not validate executionPayload and winning proposal may not execute.....	49
Informational Severity Issues.....	51
I-01: Missing createQOrg interface in IUmiaHub.....	51
I-02: QOrgToken Pause Blocks Settlement.....	52
Disclaimer.....	54
About Certora.....	54



Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Umia Chainbound	https://github.com/umia-finance/umia	(initial commit) a52d2f9107774668d2a6f f4b54e30b5e4bc5e241 (fix commit) e22f60f092ae42fa2db11 9bacc182c6d59ffe83b	Solidity

Project Overview

This document describes the security review of **Umia Chainbound**. The work was undertaken from **March 6th, 2026** to **April 8th, 2026**.

The following contract list is included in our scope:

```
smart-contracts\src\*
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Umia Finance Protocol is a protocol written in Solidity for EVM-based blockchains that enables launching on-chain organizations (called Quantum Organizations), distributing their tokens via auction, and governing their organizations and treasuries through decision markets. The smart contracts integrate with Uniswap V4 for Liquidity Launchpad and QOrg Token trading.



Launching — UmiaHub is the central registry. Calling `createQOrg` deploys a treasury contract (QOrg, behind a beacon proxy via QOrgProxy), mints the full ERC20 supply (QOrgToken), and starts a custom Liquidity Bootstrapping Pool Strategy for the auction (UmiaLBP, created by UmiaLBPFactory). When the auction (Continuous Clearing Auction) ends, `UmiaLBP.migrate()` moves liquidity into a Uniswap V4 pool, and the treasury holds the LP position NFT. After migration, trading can be paused for some time; team members can unpause early, or anyone can unpause after the deadline.

Treasury (QOrg) — Each qOrg treasury holds native ETH, ERC-20, ERC-721, and ERC-1155 assets. The founding team has no direct custody. Fund movement is gated by access control: only the governance executor can call `mint`, `burn`, `withdraw`, `withdrawERC721`, `withdrawERC1155`, and `executeCall` (arbitrary external call). During liquidation, only the authorized liquidator can withdraw. QOrg also manages a monthly allowance system: governance sets a per-token budget via `updateMonthlyAllowance`, and registered team members can draw from it each calendar month through `withdrawMonthlyAllowance`. The spent amount resets automatically on the month change. Team membership is controlled exclusively through governance (`updateTeamMember`). A document registry stores governance records on-chain. Liquidation is terminal — once `setLiquidator` is called, all treasury operations except liquidation claims are permanently blocked.

Decision market — UmiaMarketManager handles the full market lifecycle. A market creation requires an EIP-712 signature from a trusted signer. On creation, the manager removes 50% of the qOrg's Uniswap V4 LP and initializes one CPMM (**Constant Product Market Maker**) pool per proposal (governance proposals plus a no-op proposal). Users call `split()` to lock real tokens and receive virtual conditional tokens (ERC6909) across all proposals at 1:1. Users trade virtual tokens in the CPMMs via UmiaSwapRouter. After the trading window, anyone can call `settleMarket()`, during which the ConditionalMarketOracle computes per-proposal TWAPs. Settlement defaults the winner to the no-op proposal; a governance proposal becomes the winner only if it has the strictly highest TWAP among all proposals and its TWAP exceeds the no-op TWAP by at least `winningMarketThresholdBps` (basis points). If that threshold is not met (or the no-op ties or leads), the winning proposal remains the no-op — i.e., no governance execution payload is chosen for that market cycle. After that, users call `claimSettlement()` to redeem winning virtual tokens 1:1 for real tokens. `merge()` lets users exit early by burning equal amounts of virtual balance across all proposals.

Governance execution — `executeWinningProposal()` routes the winning proposal's payload to GovernanceExecutor, which decodes it and executes each action.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.



Threat and Security Overview

Threat and Security Overview

This section summarizes the threat model for the Umia protocol, focusing on treasury control, correct decision market settlement, governance-controlled token minting and burning, and access controls that prevent unauthorized actions.

Expected Main Invariants

- **QOrg Treasury contract logic:** funds can only be moved by winning proposals through the Governance executor after a properly settled decision market or pre-configured disbursement.
- **Governance Autonomy:** While the Hub facilitates creation of QOrgs, individual QOrgs should maintain a degree of isolation from Hub-level compromises
- **Access controls:** no unauthorized party can bypass governance constraints. Functions are protected with appropriate access control modifiers; unauthorized access is not possible.
- **Decision market settlement:** winning outcomes are correctly determined based on the TWAP oracle price
- **Settlement Integrity:** Market settlement must accurately reflect trading activity without dilution from non-trading periods
- **Token issuance and burning:** supply changes only occur through governance-approved proposals
- **Split/merge solvency:** real balances must always cover the maximum winning virtual supply, so claims can be paid 1:1.
- **Asset Conservation:** User-provided liquidity to Constant Product Market Maker (CPMM) must be recoverable or correctly accounted for during settlement
- **Single claim:** each user can claim settlement only once per market.
- **Single execution:** a winning proposal can only be executed once.



Trust Boundaries and Security Assumptions

- `UmiaHub.owner()` is the main trust boundary. This role controls qOrg creation, critical configuration, executor settings, oracle settings, thresholds, operator approvals, and upgrades. If this key is compromised, the attacker can affect every qOrg.
- `marketCreationSigner` is trusted to approve market definitions and payloads through EIP-712.
- `GovernanceExecutor` turns a market result into treasury actions such as `MINT_TOKENS`, `TRANSFER_TREASURY_ASSETS`, `CALL`, `LIQUIDATE_TREASURY`, and `UPGRADE_IMPLEMENTATION`.
- The protocol depends on and assumes `Uniswap V4`, `LBP/CCA`, `Permit2`, and standard tokens are secure and behave as designed.

Attack Vectors Considered

- privileged access abuse, including Hub owner compromise, malicious executor assignment, and unsafe upgrades.
- Threshold and configuration changes affect active markets
- governance payload abuse, such as duplicate assets, revert one of the actions in the winning proposal
- Value Flows & Economic Risks: accounting, solvency bugs in conditional markets from split, merge, swap, add liquidity, remove liquidity, fee accounting
- Attacks to exploit rounding issues
- oracle/TWAP risks such as Denial of service attacks, TWAP initial snapshot dilution, live oracle replacement, insufficient history, and settlement-time manipulation.
- liquidation edge cases, including assets stuck after liquidation, QOrg token paused during liquidation
- Attacks related to integration and cross-protocol interactions with UniswapV4, LBP, and CCA, such as migrations (including native ETH), LP NFT approval revocation, governance withdrawal of the LP position, permissionless migration or sweep token calls at bad times, etc.



- market creation risk, including wrong execution payload, and winning proposal may not execute
- Unexpected states, such as QOrgToken pausing can block market settlements

Highest Severity Issues and Key Security Takeaways

All contracts were manually reviewed line by line. The audit identified the high and medium severity issues, including:

H-01: Snapshots are taken at market creation rather than at trading start, diluting real market activity by 25–50%.

H-02: A flaw in settlement logic treats user-provided liquidity as protocol surplus, causing users to permanently lose **100%** of their added tokens.

M-01: **UmiaLBP** lacks a **receive()** function, causing all migration attempts to revert when using native **ETH** as the currency.

M-02: Updating a QOrg's position operator silently revokes the MarketManager's permissions, permanently freezing pending decision markets and user funds.

M-03: Liquidation is irreversible and lacks support for ERC721s, causing NFTs and any omitted ERC20 tokens to be trapped in the contract forever.

M-04: A compromised Hub owner can upgrade the Hub or swap executors to directly drain assets from any QOrg treasury.

M-05: Leftover tokens stuck permanently in UmiaMarketManager after re-adding liquidity to Uniswap V4 during market settlement

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	2	2	2
Medium	5	5	3
Low	11	11	10
Informational	2	2	2
Total	20	20	17

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				



Detailed Findings

ID	Title	Severity	Status
H-01	TWAP initial snapshot dilution	High	Fixed
H-02	User CPMM LP Permanently Lost at Settlement	High	Fixed
M-01	migrate() always fails when currency is ETH because UmiaLBP cannot accept native ETH	Medium	Fixed
M-02	Updating a QOrg's position operator via approveQOrgPositionOperator silently breaks all pending decision markets	Medium	Acknowledged
M-03	Assets might stuck permanently after liquidation starts	Medium	Fixed
M-04	A hacked Hub owner can steal assets from any QOrg	Medium	Acknowledged
M-05	Leftover tokens stuck permanently in UmiaMarketManager after re-adding liquidity to Uniswap V4 during market settlement	Medium	Fixed
L-01	Paused QOrg Token blocks liquidation claims after liquidation starts	Low	Fixed



L-02	Migration uses fixed reserveTokenAmount and ignores swept unsold tokens in calculating liquidity to initialize UniswapV4 Pool	Low	Fixed
L-03	TWAP cardinality slot filling enables permissionless DoS on market creation and settlement	Low	Fixed
L-04	LP NFT withdrawal permanently breaks market creation	Low	Fixed
L-05	Executor's `ActionType.CALL` enables unauthorized address(0) upgrade and beacon reset	Low	Fixed
L-06	Conditional market swap applies two fee layers and effective fee exceeds conditionalMarketFeeBps	Low	Fixed
L-07	Threshold and config changes affect active markets	Low	Fixed
L-08	Unbounded string lengths in `uploadDocument` allow for permanent state bloat and gas limit DoS	Low	Fixed
L-09	A single reverting action can permanently DoS an entire governance execution plan	Low	Acknowledged
L-10	Duplicate assets in `LIQUIDATE_TREASURY` payloads allow users to drain the treasury via multiple payouts	Low	Fixed



L-11	CreateMarket does not validate executionPayload and winning proposal may not execute	Low	Fixed
I-01	Missing createQOrg interface in IUmiaHub	Informational	Fixed
I-02	QOrgToken Pause Blocks Settlement	Informational	Fixed

High Severity Issues

H-01: TWAP initial snapshot dilution

Severity: High	Impact: Medium	Likelihood: High
Files: smart-contracts/src/periphery/UmiaTWAPOracle.sol	Status: Fixed	

Description:

The TWAP oracle snapshot is recorded when `createMarket()` is called, not when trading begins. The `recordSnapshotIfNeeded()` function fires during market creation:

```
IUmiaTWAPOracle oracle = IUmiaTWAPOracle(HUB.twapOracle());  
  
for (uint256 i = 0; i < proposalIds.length; i++) {  
  
    CPMM.State memory state = cpmmStates[proposalIds[i]];  
  
    oracle.update(proposalIds[i], state.reserve0, state.reserve1);  
  
    oracle.recordSnapshotIfNeeded(proposalIds[i]);  
  
}
```

The snapshot records the current cumulative price and timestamp:

```
function recordSnapshotIfNeeded(uint256 proposalId) external onlyMarketManager {  
  
    TradingStartSnapshot storage snapshot = tradingStartSnapshots[proposalId];  
  
    if (!snapshot.recorded) {  
  
        OracleState storage oracle = oracleStates[proposalId];  
  
        snapshot.price0Cumulative = oracle.price0CumulativeLast;
```



```
        snapshot.price1Cumulative = oracle.price1CumulativeLast;

        snapshot.timestamp = oracle.blockTimestampLast;

        snapshot.recorded = true;
    }
}
```

When `calculateTWAP()` runs at settlement, it computes:

```
// UmiaTWAPOracle.sol:L133-135

uint32 totalTimeElapsed;

unchecked {

    totalTimeElapsed = uint32(block.timestamp) - startSnapshot.timestamp;

}
```

This `totalTimeElapsed` includes the entire pre-trading delay period (1-3 days between creation and `tradingStart`). During this period, no trading occurs, but the creation-time price continues to accumulate in the TWAP window. The pre-trading period contributes:

- 1-day delay + 3-day trading = 25% weight for the static creation-time price
- 3-day delay + 3-day trading = 50% weight for the static creation-time price

This has two consequences:

1. Trading activity during the actual trading period is diluted by 25–50%, making the TWAP less responsive to genuine market sentiment.
1. All proposals start with identical prices (same initial reserves), so the pre-trading anchor is neutral — but it reduces the cost of manipulation because the attacker's trades only need to overcome a smaller effective window.

**Recommendations:**

Record the TWAP snapshot at `tradingStart` instead of at `createMarket()`.

Customer Response:

Fixed in [60834f76](#).

Fix Review:

Fix confirmed.



H-02: User CPMM LP Permanently Lost at Settlement

Severity: High	Impact: High	Likelihood: Medium
Files: smart-contracts/src/core/U miaMarketManager.sol	Status: Fixed	

Description:

When a user calls `addLiquidity()`, their virtual tokens are transferred from the user to the CPMM reserves and the `userVirtualSupply` trackers are decremented accordingly. However, the corresponding `realQOrgBalance` and `realMoneyBalance` trackers are never decremented. At settlement, the protocol computes the excess tokens available for re-adding to the Uniswap V4 LP position as `totalRealTokens - userVirtualSupply`. Because the real balance was not reduced but the virtual supply was, the settlement computation treats the user's LP'd tokens as surplus protocol funds and sends them to the QOrg's V4 position. The LP provider permanently loses 100% of the tokens they added as liquidity.

In `addLiquidity()`, virtual supply is reduced but real balance is untouched:

```
transferFrom(msg.sender, address(this), getVirtualQOrgId(proposalId),
result.actualAmount0);

transferFrom(msg.sender, address(this), getVirtualMoneyId(proposalId),
result.actualAmount1);

// Update user virtual supply (tokens moved from user to contract)

userVirtualQOrgSupply[proposalId] -= result.actualAmount0;

userVirtualMoneySupply[proposalId] -= result.actualAmount1;

// At settlement, the excess computation inflates by exactly the LP'd amount:
```




```
uint256 userQOrgClaims = userVirtualQOrgSupply[winningProposalId];

uint256 userMoneyClaims = userVirtualMoneySupply[winningProposalId];

uint256 totalRealQOrg = realQOrgBalance[marketId] + removalInfo.qOrgRemoved;

uint256 totalRealMoney = realMoneyBalance[marketId] + removalInfo.moneyRemoved;

uint256 qOrgToReAdd = totalRealQOrg > userQOrgClaims + 1 ? totalRealQOrg -
userQOrgClaims - 1 : 0;

uint256 moneyToReAdd = totalRealMoney > userMoneyClaims + 1 ? totalRealMoney -
userMoneyClaims - 1 : 0;

if (removalInfo.lpTokenId != 0 && (qOrgToReAdd > 0 || moneyToReAdd > 0)) {
    _reAddLiquidity(market.qOrgId, qOrgToReAdd, moneyToReAdd, marketId);
}
```

Algebraically, if a user splits S tokens and LP's L of them, the settlement sends $M + L - 1$ tokens to V4 (where M is the protocol seed from LP removal) instead of the correct $M - 1$. Only $S - L + 1$ tokens remain for claims instead of the full S . The user's loss equals exactly L — 100% of the LP'd amount.

There is no `removeLiquidity()` function, so once a user provides CPMM liquidity, the loss is guaranteed and irreversible.

Recommendations:

Track LP contributions separately and account for them during settlement. Two complementary fixes:

1. Add a `removeLiquidity()` function allowing users to withdraw LP before settlement.
1. During settlement, compute excess as `totalRealTokens - userVirtualSupply - totalLPContributions` to prevent LP'd tokens from being treated as surplus. Alternatively, reduce `realBalance` trackers when `addLiquidity()` is called.

**Customer Response:**

Fixed in [bee1128b](#).

Fix Review:

Fix confirmed.

Medium Severity Issues

M-01: migrate() always fails when currency is ETH because UmiaLBP cannot accept native ETH

Severity: Medium	Impact: High	Likelihood: Low
Files: smart-contracts/src/launchpad/UmiaLBP.sol	Status: Fixed	

Description:

UmiaLBP is designed to support native ETH as the auction currency. The code explicitly branches on native currency when transferring assets and calling the position manager:

```
// File: smart-contracts/src/launchpad/UmiaLBP.sol
function _transferAssetsAndExecutePlan(uint128 _tokenAmount, uint128
_currencyAmount, bytes memory _plan) internal {
    Currency.wrap(token).transfer(address(positionManager), _tokenAmount);
    if (Currency.wrap(currency).isAddressZero()) {
        positionManager.modifyLiquidities{value: _currencyAmount}(_plan,
block.timestamp);
    } else {
        Currency.wrap(currency).transfer(address(positionManager),
_currencyAmount);
        positionManager.modifyLiquidities(_plan, block.timestamp);
    }
}
```

When `currency` is `address(0)`, the contract sends ETH to the position manager via `{value: _currencyAmount}`. The ETH comes from the auction initializer (CCA) when `migrate()` calls `cca.sweepCurrency()`.

```
function migrate() external nonReentrant {
    // 3. Sweep CCA funds to this contract (skips if already swept)
    ContinuousClearingAuction cca =
```



```
ContinuousClearingAuction(address(initializer));  
    if (cca.sweepCurrencyBlock() == 0) {  
        cca.sweepCurrency();  
    }  
}
```

UmiaLBP does not implement `receive()` or a `payable fallback()`. So when the CCA tries to send ETH to UmiaLBP, the transfer reverts. As a result, `migrate()` always reverts.

Calling `sweepCurrency()` on the CCA contract separately also reverts, so there is no way to sweep Currency from the CCA contract.

Recommendations:

Add a `receive()` function to UmiaLBP so the contract can accept native ETH when the auction contract sweeps currency to it.

Customer Response:

Fixed in [19fd4c44](#).

Fix Review:

Fix confirmed.

M-02: Updating a QOrg's position operator via `approveQOrgPositionOperator` silently breaks all pending decision markets

Severity: Medium	Impact: High	Likelihood: Low
Files: smart-contracts\src\core\UmiaMarketManager.sol smart-contracts\src\core\UmiaHub.sol smart-contracts\src\core\QOrg.sol	Status: Acknowledged	

Description:

`UmiaMarketManager` requires `ERC721` approval to operate on a `QOrg`'s Uniswap V4 LP position NFT, tracked by `v4LiquidityTokenId`), in order to execute `removeLiquidity` and `reAddLiquidity` during the market lifecycle. This approval is initially granted during the `onLBPMigrated` callback.

However, `UmiaHub` provides a management function `approveQOrgPositionOperator` which allows the Hub owner to grant approval to a different operator address. Since `ERC721` only allows a single approved operator per `tokenId` at any time, calling this function immediately and silently revokes `UmiaMarketManager`'s permission to modify the `QOrg`'s liquidity.

If any decision markets for that `QOrg` are currently OPEN or ENDED but not yet settled, they will be permanently broken. Any attempt to call `settleMarket` will revert when it tries to `reAddLiquidity`, effectively locking all user funds and real tokens inside the market manager and preventing the execution of any governance proposals or the creation of any new markets, due to the attempt to call `removeLiquidity`.

Recommendations:

To allow the Hub owner to manage the position without breaking the protocol's internal markets, the protocol should use `setApprovalForAll` instead of `approve` which allows multiple operators to work in parallel.

Customer Response:

Acknowledged. The current code still uses `approve()` rather than `setApprovalForAll()`. In practice, `approveQOrgPositionOperator` is called only to set the `UmiaMarketManager` as the operator, and



the Hub owner would not call it during active markets. We accept this as a known operational constraint and will document it in operator runbooks.

M-03: Assets might stuck permanently after liquidation starts

Severity: Medium	Impact: High	Likelihood: Low
Files: smart-contracts/src/core/Q Org.sol	Status: Fixed	

Description:

Liquidation is one way, irreversible: after `startLiquidation()` is called on a QOrg, all ERC721 tokens held in the treasury cannot be withdrawn and are not distributed during liquidation claims. ERC721s (including the Uniswap V4 position NFT) become permanently stuck in the contract.

```
// ERC721 excluded from liquidation snapshot
if (asset.assetType == GovernanceTypes.AssetType.ERC721) revert InvalidParams();
```

Withdrawing ERC721 is blocked after liquidation, and claimLiquidation does not handle ERC721.

```
// QOrg.sol
function withdrawERC721(address _token, address _to, uint256 _tokenId)
    external
    onlyHubOrMarketManagerOrExecutor
    whenNotLiquidating // reverts if liquidationActive == true
    nonReentrant
```

The contract does not maintain an explicit list of treasury assets. Liquidation snapshots only the assets supplied in the proposal. If the winning liquidation proposal omits any asset (e.g. an ERC20 the treasury holds), that asset is also stuck forever.

Recommendations:

- Implement explicit asset management by maintaining a list of treasury assets
- Validate before liquidation that no ERC721 remained in the contract before liquidation, and all assets are included in the liquidation proposals.

Customer Response:



Partially fixed in [19fd4c44](#) and in PRs: [#438](#), [#442](#). Introduced a new SimpleLiquidator contract (`src/liquidation/SimpleLiquidator.sol`) with ILiquidator interface that maintains an explicit asset list via `_liquidationAssets[]` . Asset balances are snapshotted during `initialize()` , preventing omission of known assets. Duplicate assets are validated and rejected. However, the liquidator currently only supports NATIVE and ERC-20 assets. ERC-721 tokens (including the V4 position NFT), ERC-1155 are intentionally excluded from the liquidation flow.

Fix Review:

Fix confirmed.

M-04: A hacked Hub owner can steal assets from any QOrg

Severity: Medium	Impact: High	Likelihood: Low
Files: smart-contracts/src/core/QOrg.sol smart-contracts/src/core/UmiaHub.sol	Status: Acknowledged	

Description:

The Hub owner can upgrade the Hub itself, set the default and per-QOrg governance executor, and control other critical protocol parameters. The change takes effect immediately. With the current design, if the Hub owner is compromised, attackers can drain the tokens from any organization (QOrg) easily in at least 3 ways:

(1) Via Hub upgrade:

The Hub owner can upgrade the contract to a malicious implementation that exposes backdoor functions. Because QOrg's mint, withdraw, withdrawERC721, withdrawERC1155, and executeCall use the modifier onlyHubOrMarketManagerOrExecutor, and the Hub is explicitly allowed, a malicious implementation can call these functions on any QOrg and drain treasury assets directly.

(2) Via setQOrgGovernanceExecutor:

Hub Owner calls to `setQOrgGovernanceExecutor(qOrgId, malicious)` to set GovernanceExecutor of any QOrg to any address. Then the malicious executor can then mint, withdraw, and/or upgrade QOrg. This update is made without the QOrg on-chain agreement/approval proposal.

(3) Via upgrade UmiaMarketManager

The Hub owner can upgrade the UmiaMarketManager to a malicious implementation.

Although the protocol may rely on legally binding (e.g., off-chain agreements, shared responsibility with launching projects), if a QOrg holds a treasury with significant value in the

QOrg contract, this design is problematic and hard to accept for many projects and users, so this can be a barrier to the adoption of the protocol.

Recommendations:

- Remove Hub permission from QOrg functions to mint, burn, withdraw assets, and executeCall by modifying the modifier `onlyHubOrMarketManagerOrExecutorUmiaHub` to exclude Hub, since the Hub permission is not necessary.
- Hub Implementation upgrade, UmiaHubMarketManager upgrade, and `setQOrgGovernanceExecutor` transaction: implement a Timelock mechanism to execute these transactions with a reasonable delay (e.g., 7 days)
- Consider implementing a 2-step approval process with QOrg approval via a winning governance proposal: `setQOrgGovernanceExecutor` transaction sets the pending Executor for the QOrg. Then, in a QOrg's winning proposal, the QOrg contract can accept the pending Executor to be the official GovernanceExecutor.

Customer Response:

Acknowledged. Timelock addition is WIP. We will use standard timelocks from OpenZeppelin here.

M-05: Leftover tokens stuck permanently in UmiaMarketManager after re-adding liquidity to Uniswap V4 during market settlement

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: smart-contracts/src/core/UmiaMarketManager.sol smart-contracts/src/libraries/UniswapHelperLib.sol	Status: Fixed	

Description:

When a market is settled via `settleMarket()`, the contract calculates excess QOrg and Money tokens not needed for user claims and passes them to `_reAddLiquidity()` to deposit back into the Uniswap V4 spot pool.

```
// File: smart-contracts/src/core/UmiaMarketManager.sol
function _reAddLiquidity(address qOrgAddr, uint256 qOrgToReAdd, uint256
moneyToReAdd, uint256 marketId) internal {
    // ...
    uint128 liquidityAdded = UniswapHelperLib.reAddLiquidity(params);
    emit LiquidityReAdded(marketId, lpTokenId, liquidityAdded);
}
```

Inside `UniswapHelperLib.reAddLiquidity()`, the function first computes the maximum liquidity achievable from the available amounts, then derives the actual token amounts `amount0Used` and `amount1Used` to achieve that liquidity.

```
// File: smart-contracts/src/libraries/UniswapHelperLib.sol
uint128 maxLiquidity = LiquidityAmounts.getLiquidityForAmounts(
    sqrtPriceX96, sqrtLower, sqrtUpper, _params.amount0Available,
    _params.amount1Available
);

(uint256 amount0Used, uint256 amount1Used) =
    LiquidityAmounts.getAmountsForLiquidity(sqrtPriceX96, sqrtLower, sqrtUpper,
```

```
maxLiquidity);
```

```
AddLiquidityParams memory addParams = AddLiquidityParams({
    positionManager: _params.positionManager,
    permit2: _params.permit2,
    tokenId: _params.tokenId,
    amount0: amount0Used,
    amount1: amount1Used,
    amount0Max: _params.amount0Available,
    amount1Max: _params.amount1Available
});
```

The `amount0Used` and `amount1Used` will almost always be less than `amount0Available` and `amount1Available`, as seen in the function `getLiquidityForAmounts`. For the full-range liquidity position, the `sqrtPriceX96` is always in between `sqrtPriceAX96` and `sqrtPriceBX96`, so `maxLiquidity` is the minimum amount of `liquidity0` and `liquidity1`

```
function getLiquidityForAmounts(
    uint160 sqrtPriceX96,
    uint160 sqrtPriceAX96,
    uint160 sqrtPriceBX96,
    uint256 amount0,
    uint256 amount1
) internal pure returns (uint128 liquidity) {

    if (sqrtPriceX96 <= sqrtPriceAX96) {
        liquidity = getLiquidityForAmount0(sqrtPriceAX96, sqrtPriceBX96,
amount0);
    } else if (sqrtPriceX96 < sqrtPriceBX96) {
        uint128 liquidity0 = getLiquidityForAmount0(sqrtPriceX96,
sqrtPriceBX96, amount0);
        uint128 liquidity1 = getLiquidityForAmount1(sqrtPriceAX96,
sqrtPriceX96, amount1);

        liquidity = liquidity0 < liquidity1 ? liquidity0 : liquidity1; //
@note max_liquidity is minimum amount
    } else {
        liquidity = getLiquidityForAmount1(sqrtPriceAX96, sqrtPriceBX96,
amount1);
```



```
}  
}
```

Neither `reAddLiquidity(params)` nor the caller `_reAddLiquidity()` in `UmiaMarketManager` returns or tracks the unused token amounts. The Uniswap V4 `SETTLE_PAIR` action only pulls the amounts actually owed for the liquidity increase. So the leftover tokens remain stuck in the `UmiaMarketManager` contract.

The ratio of `qOrgToReAdd` to `moneyToReAdd` is shaped by the decision market trading activity. It is likely that there are leftover tokens after re-adding liquidity to Uniswap V4. There is no sweep, rescue, or withdrawal function in `UmiaMarketManager` that could recover these leftover tokens. They are permanently locked in the contract.

Recommendations:

`reAddLiquidity()` returns the actual leftover unused tokens after re-adding liquidity. The contract `UmiaMarketManager` then transfers the leftover tokens to the `QOrg` contract.

Customer Response:

Fixed in [6f27b49a](#).

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: Paused QOrg Token blocks liquidation claims after liquidation starts

Severity: Low	Impact: Medium	Likelihood: Low
Files: smart-contracts/src/core/QOrg.sol smart-contracts/src/tokens/QOrgToken.sol	Status: Fixed	

Description:

In a scenario where `QOrgToken` is paused via governance after trading starts (`tradingPauseDeadline` is 0) and the executor calls `startLiquidation`, then users' calls `claimLiquidation` always revert. Because the claim flow calls `QOrgToken.burn(msg.sender, amount)` and `QOrgToken` uses OpenZeppelin `ERC20Pausable`, so `_burn` goes through `_update`, which reverts while the token is paused. After liquidation starts, `liquidationActive` remains true forever, blocking `executeCall` and preventing governance from unpausing the token. As a result, treasury assets may be stuck in the contract.

Recommendations:

Add a check in the function `startLiquidation` to make sure that `QOrgToken` is not paused.

Customer Response:

Fixed in [2b02dd8a](#).

Fix Review:

Fix confirmed.

L-O2: Migration uses fixed reserveTokenAmount and ignores swept unsold tokens in calculating liquidity to initialize UniswapV4 Pool

Severity: Low	Impact: Low	Likelihood: Medium
Files: smart-contracts/src/launchpad/UmiaLBP.sol	Status: Fixed	

Description:

UmiaLBP stores the `reserveTokenAmount` as an immutable value in the constructor, equal to the QOrg token amount remaining in the UmiaLBP contract after transferring the token to the Auction.

During `migrate()`, the contract calls `sweepUnsoldTokens()` and can receive extra unsold QOrg tokens from the auction. However, when `_prepareMigrationData()` calculates pool amounts, it uses the fixed `reserveTokenAmount` as the token cap.

```
// File: smart-contracts/src/launchpad/UmiaLBP.sol
function _prepareMigrationData(uint128 currencyAmount, uint256 initialPriceX96)
    internal
    view
    returns (MigrationData memory)
{
    // Redacted
    (uint128 fullRangeTokenAmount, uint128 fullRangeCurrencyAmount) =
        priceX192.calculateAmounts(currencyAmount, currencyIsCurrency0,
reserveTokenAmount);

    // Redacted

}
```

In `TokenPricing.calculateAmounts()`, if the computed token amount is above this cap, the token amount is clamped to `reserveTokenAmount`. So, unsold tokens are not considered in the migration amount calculation. As a result, in some cases, the UniswapV4 pool can be initialized

with less token liquidity than possible, and extra tokens are later sent to [qorg](#) as leftovers. So this makes capital usage during the migration less efficient.

Recommendations:

Use the real token balance after [sweepUnsoldTokens](#) when calculating migration amounts

Customer Response:

Fixed in [19fd4c44](#).

Fix Review:

Fix looks good.

L-03: TWAP cardinality slot filling enables permissionless DoS on market creation and settlement

Severity: Low	Impact: Low	Likelihood: Medium
Files: smart-contracts\src\periphery\SpotMarketPriceGuard.sol smart-contracts\src\libraries\SpotMarketOracle.sol smart-contracts\src\core\UmiMarketManager.sol	Status: Fixed	

Description:

The `SpotMarketPriceGuard.checkDeviation()` protects `createMarket` and `settleMarket` against spot-price manipulation by querying a 4-hour TWAP from the `SpotMarketOracle`. The oracle stores observations in a circular buffer initialized to 250 slots during LBP migratio. A critical constraint applies: the oracle writes at most one observation per block. This means the buffer's time coverage is determined by block time, not swap count.

This 250-slot buffer is fundamentally undersized relative to the 4-hour TWAP window (both derived from scripts) for any realistic chain. On L1 (12s blocks), 250 observations span only ~50 minutes. On L2s like Base (2s blocks), they span only ~500 seconds (~8.3 minutes). Since the guard is fail-closed, any gap between the buffer's time coverage and the TWAP window produces a revert rather than a bypass.

When trading activity is low, observations age naturally. If swaps are infrequent enough that the oldest observation in the 250-slot buffer is older than the TWAP window (4 hours), `observe(TWAP_WINDOW)` succeeds, the guard computes the TWAP tick, compares it to the spot tick, and allows or rejects the operation based on deviation. However, an attacker can target a specific `qOrg`'s spot pool by executing one micro-swap per block across 250 consecutive blocks. Each swap advances the circular buffer index and overwrites the oldest observation. After 250 blocks of continuous single-swap activity, every slot in the buffer holds a recent observation — the oldest is only $250 \times \text{block_time}$ seconds old.

When `SpotMarketPriceGuard.checkDeviation()` then calls `lbp.observe(key, [TWAP_WINDOW, 0])`, the oracle's `getSurroundingObservations()` finds that the target timestamp `now - 14400` predates the oldest buffered observation and reverts. The guard's catch block then confirms the oracle IS initialized by successfully calling `observe(0)`, and reverts with `InsufficientOracleHistory`:

```
catch {  
  
    uint32[] memory current = new uint32[](1);  
  
    current[0] = 0;  
  
    try lbp.observe(key, current) {  
  
        revert InsufficientOracleHistory(); // Oracle active but buffer evicted  
  
    } catch {  
  
        // Oracle truly uninitialized (pre-migration) – silently allow  
  
    }  
  
}
```

The attack is self-healing: if the attacker stops, the buffer ages naturally and the guard recovers after `TWAP_WINDOW` seconds of inactivity. However there is also a natural lockout – after LBP migration, the oracle is initialized with a single observation at the migration timestamp. The guard's `observe(14400)` call immediately fails because the pool is less than 4 hours old. The catch block confirms initialization and reverts with `InsufficientOracleHistory`. Every new `qOrg` is therefore blocked from creating its first market for exactly `TWAP_WINDOW` seconds after migration, regardless of attacker presence.

This is low cost to execute from a single malicious entity and blocks both `createMarket` and `settleMarket` directly, which indirectly blocks `claimSettlement`

Recommendations:

When `observe(TWAP_WINDOW)` fails, instead of immediately reverting, query the oracle for the oldest available observation and use whatever history exists. If the available window is shorter



than `TWAP_WINDOW` but still above a minimum threshold (e.g., 30 minutes), perform the TWAP deviation check with the shorter window. Only revert if available history is below the minimum. Consider increasing the default cardinality for good measure instead of relying solely on `increaseCardinalityNext` and changing the TWAP writes from per-block to a time based, for example every minute, which for 250 slots * 60 seconds covers more than 4 hours.

Customer Response:

Fixed in [b5115fcc](#).

Fix Review:

Fix looks good.



L-04: LP NFT withdrawal permanently breaks market creation

Severity: Low	Impact: Medium	Likelihood: Low
Files: smart-contracts\src\core\Q Org.sol smart-contracts\src\libraries \GovernanceVM.sol smart-contracts\src\core\U miaMarketManager.sol	Status: Fixed	

Description:

The **QOrg**'s **withdrawERC721()** function has no exclusion for the Uniswap V4 LP position NFT that is critical for market creation:

```
function withdrawERC721(address _token, address _to, uint256 _tokenId)

    external

    onlyHubOrMarketManagerOrExecutor

    whenNotLiquidating

    nonReentrant

{

    IERC721(_token).safeTransferFrom(address(this), _to, _tokenId);

}
```

A winning governance proposal can encode either a **TRANSFER_TREASURY_ASSETS** action with **AssetType.ERC721** targeting the LP NFT, or a **CALL** action to transfer it. Once the NFT is transferred out, **createMarket()** permanently reverts because it attempts to remove liquidity from an NFT the **QOrg** no longer owns:



```
uint256 lpTokenId = IQOrg(qOrgInfo.qOrg).v4LiquidityTokenId();

if (lpTokenId == 0) revert LPPositionNotRegistered();

(, uint128 liquidity, uint256 amount0, uint256 amount1) =
getPositionAmounts(lpTokenId);
```

The `v4LiquidityTokenId` state variable retains the old token ID even after the NFT is gone (it is never cleared), and `onLBPMigrated` blocks re-registration with `if (v4LiquidityTokenId != 0) revert LPAlreadyRegistered()`. This creates an irrecoverable state.

Thus a single governance proposal can permanently disable all future decision markets for a `QOrg` by withdrawing the LP position NFT. The `v4LiquidityTokenId` cannot be reset, `onLBPMigrated` cannot be called again, and all `createMarket()` calls will revert. Recovery requires a `QOrg` implementation upgrade (via beacon or UUPS), which may itself depend on governance that is now broken.

Recommendations:

Add an exclusion check in `withdrawERC721()` that prevents transferring the LP position NFT. Alternatively, add a recovery mechanism to reset `v4LiquidityTokenId` in case of accidental or malicious withdrawal, callable by the Hub owner.

Customer Response:

Fixed in [e6d14c59](#).

Fix Review:

Fix confirmed.

L-05: Executor's `ActionType.CALL` enables unauthorized address(0) upgrade and beacon reset

Severity: Low	Impact: Low	Likelihood: Low
Files: smart-contracts\src\libraries \GovernanceVM.sol	Status: Fixed	

Description:

The **GovernanceVM** library contains a logic flaw where the **ActionType.CALL** action can be used to bypass security checks implemented for other action types. Specifically, an attacker (or a malicious/erroneous governance proposal) can use **ActionType.CALL** to target the proxy itself and invoke **upgradeToAndCall(address(0), "")**. This bypasses the explicit **address(0)** check in the **UPGRADE_IMPLEMENTATION** handler and forces the proxy to reset its implementation slot, effectively opting back into beacon-based upgrades without following the intended upgrade validation path.

```
if (actionType == GovernanceTypes.ActionType.UPGRADE_IMPLEMENTATION) {  
    GovernanceTypes.UpgradeImplementation memory params = abi.decode(  
        action.data, (GovernanceTypes.UpgradeImplementation)  
    );  
  
    if (params.newImplementation == address(0)) revert InvalidParams(); // <---  
Check  
  
    UUPSUpgradeable(address(qOrg)).upgradeToAndCall(  
        params.newImplementation, params.data  
    );  
  
    // ...
```



```
}
```

However, the CALL action type allows arbitrary calls to any target:

```
if (actionType == GovernanceTypes.ActionType.CALL) {  
    GovernanceTypes.Call memory params = abi.decode(  
        action.data,  
        (GovernanceTypes.Call)  
    );  
    if (params.target == address(0)) revert InvalidParams();  
    qOrg.executeCall(params.target, params.value, params.data); // <--- Arbitrary  
call  
    return;  
}
```

If a proposal is created with `ActionType.CALL` where target is the `qOrg` address and the data is the encoded call to `upgradeToAndCall(address(0), "")`, the `qOrg` will execute a self-call.

This results in the `ERC1967Utils.setImplementation(address(0))` being called, which clears the implementation slot.

Recommendations:

Implement a restriction in `GovernanceVM.sol` to prevent `ActionType.CALL` from targeting the `qOrg` itself for sensitive administrative functions.

Customer Response:

Fixed in [2aa816db](#)



Fix Review:

Fix confirmed.

L-06: Conditional market swap applies two fee layers and effective fee exceeds conditionalMarketFeeBps

Severity: Low	Impact: Low	Likelihood: Medium
Files: smart-contracts/src/core/U miaMarketManager.sol smart-contracts/src/libraries /CPMMWithFee.sol smart-contracts/src/libraries /CPMM.sol	Status: Fixed	

Description:

Swaps on conditional markets charge two fee layers:

1. Protocol fee: `HUB.conditionalMarketFeeBps()` — deducted in `CPMMWithFee._swapExactIn` from input before calling CPMM; accrued to `accruedFeeQOrg` / `accruedFeeMoney`.

```
// File: smart-contracts/src/libraries/CPMMWithFee.sol
function _swapExactIn(
    CPMM.State storage state,
    uint256 amountIn,
    uint256 amountOutMin,
    uint256 maxPriceImpactBps,
    bool zeroForOne,
    uint256 protocolFeeBps
) internal returns (CPMM.SwapResult memory result, uint256 protocolFee) {
    if (protocolFeeBps > 0) {
        protocolFee = (amountIn * protocolFeeBps) / FEE_BPS_DENOM;
    }
    result = CPMM._swapExactIn(state, amountIn - protocolFee, amountOutMin,
maxPriceImpactBps, zeroForOne);
    result.amountIn = amountIn;
}
```

1. CPMM fee: `CPMM.FEE_BPS = 30` (0.3%) — applied inside `CPMM._getAmountOut`; reduces output via $\text{amountInWithFee} = \text{amountIn} * (\text{FEE_BPS_DENOM} - \text{FEE_BPS}) / \text{FEE_BPS_DENOM}$.

```
// File: smart-contracts/src/libraries/CPMM.sol
function _getAmountOut(uint256 amountIn, uint256 reserveIn, uint256
reserveOut)
    internal
    pure
    returns (uint256 amountOut)
{
    // x*y=k with fee on input
    if (reserveIn == 0 || reserveOut == 0) revert InsufficientReserves();
    uint256 amountInWithFee = (amountIn * (FEE_BPS_DENOM - FEE_BPS)) /
FEE_BPS_DENOM;
    uint256 numerator = amountInWithFee * reserveOut;
    uint256 denominator = reserveIn + amountInWithFee;
    amountOut = numerator / denominator;
}
```

The protocol fee is configurable and documented in UmiaHub; the 0.3% CPMM fee is hardcoded and not configurable. If the protocol sets “1% fee” via `conditionalMarketFeeBps = 100`, users may reasonably assume total swap fee is 1%. In reality, they pay 1% protocol fee plus 0.3% CPMM fee, so output is lower than a “1% only” expectation. Without clear documentation or an explicit “effective total fee”, this leads to an expectation mismatch.

Recommendations:

Fix the issue so that the charged fee matches `conditionalMarketFeeBps` or document clearly about the total fee is `conditionalMarketFeeBps` in UmiaHub plus 0.3% (hardcoded)

Customer Response:

Fixed in [6f8581d](#).

Fix Review:

Fix confirmed.



L-07: Threshold and config changes affect active markets

Severity: Low	Impact: Medium	Likelihood: Low
Files: smart-contracts/src/core/U miaMarketManager.sol smart-contracts/src/core/U miaHub.sol	Status: Fixed	

Description:

Hub-level configuration parameters are read dynamically at every lifecycle step rather than being snapshotted at market creation. The most impactful is `winningMarketThresholdBps`, which determines whether a proposal's TWAP advantage over the no-op is sufficient to win. The threshold is read live during `settleMarket()`:

```
if (priceDeltaBps >= HUB.winningMarketThresholdBps()) { //<--- current live
variable
    winningProposalId = highestProposalId;
    winningTWAP = highestTWAP;
}
```

The setter validates only an upper bound:

```
function setWinningMarketThresholdBps(uint16 bps) external onlyOwner {
    if (bps > MAX_BPS) revert InvalidBps();
    uint16 oldBps = winningMarketThresholdBps;
    winningMarketThresholdBps = bps;
    emit WinningMarketThresholdUpdated(oldBps, bps);
}
```



```
}
```

Similarly, all Hub address setters (`setSwapRouter`, `setTwapOracle`, `setUmiaMarketManager`, `setDefaultGovernanceExecutor`) operate independently of active market state. None of these setters check whether any QOrg has an active unsettled market before applying the change.

A couple of impacts:

- Threshold change: A market created under a 2% threshold can be settled under 0% or 50%. At threshold 0, any proposal with TWAP marginally above the no-op wins trivially. At 100%, governance is effectively frozen. The Hub owner can retroactively change the outcome of already-traded markets.
- Oracle change: A new oracle has no cumulative price data for existing proposals. Settlement produces incorrect TWAP values, potentially selecting the wrong winning proposal.
- Router change: Users cannot trade in active markets because the router-gated functions reject the old router.
- MarketManager change: The old `MarketManager` holds all virtual token state and real token balances. Active markets become unseizable, and escrowed tokens are stranded.

Recommendations:

Snapshot critical parameters at market creation time and use the snapshotted values during settlement. At minimum, snapshot `winningMarketThresholdBps` in the `Market` struct. For address parameters, add an active-market guard to all Hub setters that checks `activeMarketByQOrg` across all QOrgs before allowing changes, or consider a two-phase config update.

Customer Response:

Fixed in [262613ae](#).

Fix Review:

Fix confirmed.

L-08: Unbounded string lengths in `uploadDocument` allow for permanent state bloat and gas limit DoS

Severity: Low	Impact: Medium	Likelihood: Low
Files: smart-contracts/src/core/QOrg.sol smart-contracts/src/libraries/GovernanceVM.sol smart-contracts/src/core/GovernanceExecutor.sol	Status: Fixed	

Description:

The `QOrg.uploadDocument` function allows a governance-approved proposal to register a document's `name` and `uri`. However, the function lacks any length validation for these `string` parameters. Because `string` data is stored in the contract's state, a malicious or accidental proposal can include extremely large strings (several megabytes). This creates three distinct risks:

1. Once a large string is stored in the mapping, it remains in the Ethereum state indefinitely. There is no mechanism in `QOrg` to delete or truncate existing documents. This forces all full nodes to store the "junk" data and significantly increases the cost of future state transitions for the contract.
2. During governance execution, `GovernanceVM` must `abi.decode` these strings and pass them to the `QOrg`. The EVM's memory expansion cost grows quadratically. A "Gas Bomb" proposal containing multiple megabytes of document data will exceed the block gas limit, rendering the entire governance plan and all subsequent actions in that proposal unexecutable. This isn't fixed by the `try/catch` approach from `*include final issue*` since decoding happens before the loop and even if we `catch` the OOG revert, it would have still exhausted 63/64 of the transaction gas, surely failing by the end of the entire proposal.
3. Services that monitor `DocumentUploaded` events will be forced to process these massive strings. This can lead to OOM (Out-of-Memory) errors, timeouts, or increased infrastructure costs.

**Recommendations:**

Enforce strict length limits on both `name` and `uri` within `QOrg.uploadDocument` along reasonable bounds for the protocol's operational needs.

Customer Response:

Fixed in [92f67e4c](#).

Fix Review:

Fix confirmed.

L-09: A single reverting action can permanently DoS an entire governance execution plan

Severity: Low	Impact: Low	Likelihood: Low
Files: smart-contracts\src\core\GovernanceExecutor.sol smart-contracts\src\libraries\GovernanceVM.sol	Status: Acknowledged	

Description:

The **GovernanceExecutor** processes all actions within a governance plan sequentially in a single transaction. If any action contains a malicious revert – such as an ERC721 or ERC1155 transfer to a contract that reverts in its **onReceived** callback – the entire execution transaction will fail.

Because the **UmiaMarketManager** marks high-level execution attempts as failed via transaction revert, the **marketExecuted** flag is never set. However, if the payload contains a persistent revert vector, the proposal remains "governance-stuck." While traders can still claim their market profits, the intended organizational actions (treasury spends, upgrades, etc.) are permanently paralyzed. This creates a "Ghost Governance" scenario where market decisions have no physical impact on the protocol. This can also include any attempts to liquidate.

Recommendations:

Implement a **try/catch** block around the execution of individual actions within the **GovernanceExecutor** loop. If an action fails, emit a failure event and continue to the next action. This prevents a single malicious or broken action from holding the entire organization's governance hostage.

Customer Response:

Acknowledged. The governance executor still processes actions sequentially without try/catch wrapping. We consider this an acceptable tradeoff: wrapping individual actions in try/catch would allow partial execution of governance plans, which could leave the QOrg in an inconsistent state (e.g., upgrading the implementation but failing to set a required config). The payload validation added for L-11 mitigates the most common vectors (malformed payloads), and the



market manager now emits `GovernanceActionFailed` events for observability. Proposals are validated upfront during market creation, reducing the risk of runtime reverts.

In the future, we could introduce another action param, such as `canFail: bool`, to make actions failable at submission time. Right now, for simplicity, we would like to keep the actions as they are and default to making all of them retryable by resubmitting the transaction.

L-10: Duplicate assets in `LIQUIDATE\_TREASURY` payloads allow users to drain the treasury via multiple payouts

Severity: Low	Impact: High	Likelihood: Low
Files: smart-contracts\src\core\Q Org.sol smart-contracts\src\libraries \GovernanceVM.sol	Status: Fixed	

Description:

In `QOrg`, the `startLiquidation` function iterates through a provided array of `LiquidationAsset` structures, snapping the balance of each specified asset and pushing it to the `liquidationAssets` state array.

However, `startLiquidation` (and the `GovernanceVM.executeActionV1` logic that triggers it) inherently lacks any robust validation ensuring the uniqueness of the elements within the `_assets` array.

If an execution payload maliciously (or accidentally) specifies the exact same asset multiple times (e.g. tracking `NATIVE` twice), the contract will snap the total current balance and push it sequentially into the tracking array multiple times.

During `claimLiquidation`, the contract sequentially iterates through the `liquidationAssets` array and performs a pro-rata payout per element. If a single asset is recorded twice, a redeeming user will receive exactly `2x` their fair share of that specific asset's balance, burning their token amount only once. Consequently, the first users to claim can completely drain the contract's actual holdings, leaving subsequent users with guaranteed transaction failures.

The only actual protection against this is that the execution of the attack needs to pass through a proposal, decreasing the likelihood substantially.

**Recommendations:**

In `startLiquidation`, implement a tracking mechanism to strictly guarantee that no duplicate assets exist. If an asset type and token address combination is repeated, revert the transaction immediately or skip the duplicate.

Customer Response:

Fixed in [2aa816db](#).

Fix Review:

Fix confirmed.



L-11: CreateMarket does not validate executionPayload and winning proposal may not execute

Severity: Low	Impact: Medium	Likelihood: Low
Files: smart-contracts/src/core/U miaMarketManager.sol smart-contracts/src/core/G overnanceExecutor.sol	Status: Fixed	

Description:

In `createMarket`, each proposal's `executionPayload` is stored without proper validation (no `planVersion` check, no `ExecutionPlanV1` decode, no `validatePlanV1`). Validation only happens in `executeWinningProposal` → `GovernanceExecutor.executeProposal`. If a proposal is created with an invalid or unsupported payload (wrong plan or action version, invalid actions), the market can be created and settled, and users can `claimSettlement` normally, but the winning proposal can never be executed.

When `executeWinningProposal(marketId)` is called, it eventually invokes `GovernanceExecutor.executeProposal(..., bytes calldata executionPayload)`. There, the payload is validated:

```
if (executionPayload.length == 0) revert InvalidPayload();
if (planVersion != 1) revert GovernanceVM.InvalidVersion();
GovernanceVM.validatePlanV1(plan);
```

In `executeActionsV1` function in `smart-contracts/src/libraries/GovernanceVM.sol`

```
if (action.actionVersion != 1) revert InvalidActionVersion();

if (actionType == GovernanceTypes.ActionType.UPDATE_PARAMS) {
    revert UnsupportedAction();
}
```

**Recommendations:**

Validate `executionPayload` at market creation so that invalid payloads are rejected early.

Customer Response:

Fixed in [2aa816db](#).

Fix Review:

Fix confirmed.



Informational Severity Issues

I-01: Missing createQOrg interface in IUmiaHub

Files:

smart-contracts/src/interfaces/IUmiaHub.sol

Description:

Missing createQOrg interface in IUmiaHub

Recommendations:

Add createQOrg interface to IUmiaHub

Customer Response:

Fixed in [19fd4c44](#).

Fix Review:

Fix confirmed.

I-02: QOrgToken Pause Blocks Settlement

Files:

smart-contracts\src\core\QOrg.sol
smart-contracts\src\core\UmiaMarketManager.sol

Description:

The QOrgToken inherits from OpenZeppelin's [ERC20Pausable](#). When paused, all token transfers revert. The [claimSettlement\(\)](#) function transfers real QOrg tokens to the user:

```
address qOrgToken = IQOrg(qOrgInfo.qOrg).token();  
  
IERC20(qOrgToken).safeTransfer(msg.sender, virtualQOrgBalance);
```

If the QOrgToken is paused at the time of settlement claims, this transfer reverts, blocking all users from claiming their settlement proceeds. The token can be paused through governance action (a winning proposal with a CALL action targeting [QOrgToken.pause\(\)](#)), or during the initial LBP migration at [QOrg.sol](#):

```
if (tradingPauseDuration > 0) {  
  
    tradingPauseDeadline = block.timestamp + tradingPauseDuration;  
  
    QOrgToken(token).pause();  
  
}
```

While the initial pause has a deadline and [startTrading\(\)](#) can unpause it, a governance-initiated pause through the CALL action has no built-in unpause mechanism beyond creating another governance market to unpause, which itself requires trading the QOrgToken, creating a deadlock.

Recommendations:

Add a [QOrgToken](#) pause bypass for settlement claims. One approach is to have `claimSettlement()` transfer money tokens immediately and escrow QOrg tokens in a separate claim contract that does not depend on the token's pause state. Alternatively, add an emergency



unpause mechanism callable by the Hub owner that cannot be blocked by governance in case such a deadlocking proposal passes.

Customer Response:

Fixed in [2b02dd8a](#).

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.